Lesson notes · for you

Finding groups

Data science · Level 4 · Lesson 2

Time
30 min

Materials
A notebook and a pencil; A phone or tablet with Darsi, for the lab

Learning goals

- Tell clustering from classification.
- Run k-means steps.
- Use groups with care.

Activity: Human k-means

1. Warm-up (5 min): Draw ten dots in two loose clumps in your notebook, swap with a partner and circle any groups you see.
2. Move (10 min): Two classmates stand as centres and everyone walks to the nearer one.
3. Repeat (10 min): Each centre moves to the middle of its group, and everyone checks again.
4. Wrap-up (5 min): Say your finding in one sentence with a number in it, then fill in the exit ticket.

Discussion

- Did anyone change group?
- Does clustering need labels?
- Why choose neutral names?

Check yourself

- Centres at 3 and 10. Which centre does a pupil with 6 loans join?

Try it this week
Plot the sleep and screen time of a few friends, without names, and look for groups. Give each group a kind, neutral name.

Worksheet

Finding groups

Data science · Level 4 · Lesson 2

Name _____ Class _____ Date _____

1 **Does clustering need examples that already have labels?**

◯ Only for k = 1 ◯ Yes, always ◯ No, it finds groups on its own

2 **Order one run of k-means.**

☐ Assign points to the nearest centroid

☐ Choose k and initial centroids

☐ Repeat until assignments stop changing

☐ Move centroids to their means

3 **Run one k-means step in 1-D with centroids 2 and 5, and print the new centroids.**

```
x = [1, 2, 3, 8, 9, 10]
c = [2, 5]
g = [[], []]
print(c)
```

Write your program:

4 **k-means always finds the globally best clustering.**

◯ True ◯ False

5 **Within-cluster error is 100, 40, 32, 29 for k = 1, 2, 3, 4. Which k does the elbow suggest?**

◯ 4 ◯ 1 ◯ 2

✎ **Apps put people into groups to show them adverts. Which group do you think an app has put you in?**

Check your answers

Finding groups

Data science · Level 4 · Lesson 2

1 **No, it finds groups on its own**

Clustering works without any labels.

2 **Top to bottom: 2, 1, 4, 3**

Assignment and update alternate until convergence.

3

```
x = [1, 2, 3, 8, 9, 10]
c = [2, 5]
g = [[], []]
for v in x:
    d0 = abs(v - c[0])
    d1 = abs(v - c[1])
    k = 0 if d0 <= d1 else 1
    g[k].append(v)
print([sum(a) / len(a) for a in g])
```

1, 2, 3 go to centroid 2 and 8, 9, 10 to centroid 5; the update moves them to 2.0 and 9.0.

4 **False**

It converges to a local optimum that depends on the start.

5 **2**

The big drop is from 1 to 2; after that the gains are small.